ECEC661 — Homework 3

Custom AXI4-Lite IP Design: Fibonacci Generator and Fixed-Point Accumulator

Gary Pham — gp492

April 23, 2026

Abstract

This report documents two custom AXI4-Lite IP blocks developed in Vivado 2025.2 and exercised in bare-metal Vitis applications on the Digilent Cora Z7-07S board. The first IP, `fib_axi`, computes the N-th Fibonacci number $F(N)$ in hardware using an iterative add-and-shift FSM with overflow detection on a 32-bit unsigned result. The second IP, `fix_acc_axi`, is a fixed-point accumulator that uses the Vivado IP-Catalog `c_accum_0` block wired so that every AXI write advances $Q \leftarrow Q + B$. Both IPs were verified (a) in simulation with self-checking VHDL testbenches and (b) on hardware by driving them from a C application and capturing the UART output in PuTTY. For each design the user-logic HDL, a testbench/block-diagram snippet, the test application C code, and the captured PuTTY output are presented below.

1 Problem Statement

Design two custom AXI4-Lite peripherals and demonstrate that a Zynq-7000S Processing System (PS) can drive each of them from a bare-metal C application:

- **Problem 8.7.1 — Fibonacci Number Generator.** Given an input N, the IP computes $F(N)$ where $F(0)$=0, $F(1)$=1, $F(n)$=$F(n-1)$ + $F(n-2)$. The result is a 32-bit unsigned value with explicit overflow detection; $F(47)$ is the largest index that fits in 32 bits.
- **Problem 8.7.2 — Fixed-Point Accumulator IP.** A 32-bit signed accumulator $Q \leftarrow Q + B$ with a synchronous clear, wrapped in AXI4-Lite so that every register write advances the accumulator by exactly one step. Must reproduce the simulation figure from the course text (Digital Systems Projects, Fig. 8.7.3).

Deliverables (per design).

- User-logic HDL source code.
- Simulation / test-bench block diagram snip.
- Test application C code.
- PuTTY UART output snip.

2 Design A — Fibonacci Generator (fib_gen)

2.1 User-Logic HDL

The user logic is split into a compute core (`fib_core`) and an AXI4-Lite slave wrapper (`fib_axi_v1_0_S00_AXI`). The core implements a three-state FSM (`S_IDLE`, `S_INIT`, `S_ADD`). On a `start` pulse the core latches N, initialises (a, b, k)=$(0, 1, 1)$, and iterates the recurrence on every clock edge until k+1 = N or a 33-bit carry-extended adder flags overflow. `done`, `overflow` and `busy` are latched status flags that are cleared on the next `start`.

The AXI slave exposes four word registers: `CTRL_STAT` (START write, DONE/OVERFLOW/BUSY read) at `0x00`, `N_REG` at `0x04`, `FIB_REG` at `0x08`, and `CYCLES` at `0x0C`. Listing 1 is the full user-logic core; the AXI wrapper is omitted for brevity and submitted alongside the report as `fib_axi_v1_0_S00_AXI.vhd` and `fib_axi_v1_0.vhd`.

```
-------------------------------------------------------------------------------
-- fib_core.vhd
--
-- ECEC 661 - Homework 3 - Problem 8.7.1 Fibonacci Number Generator
--
-- Sequential Fibonacci core. Given an input N, computes F(N) where
--     F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2).
-- 32-bit unsigned result. A 33-bit adder is used to detect unsigned
-- overflow; the 'overflow' flag sticks until the next start pulse.
--
-- Handshake:
--   * Host drives N, asserts start for one cycle.
--   * Core deasserts done, asserts busy, runs its FSM.
--   * When finished, busy returns low and done stays high (latched)
--     until the next start pulse.
--
-- F(47) = 2_971_215_073 is the largest value that fits in 32 bits
-- unsigned; F(48) triggers overflow.
--
-------------------------------------------------------------------------------

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity fib_core is
    port (
        clk      : in  std_logic;
        rst      : in  std_logic;                      -- synchronous, active high
        start    : in  std_logic;                      -- 1-cycle pulse from AXI
        n        : in  std_logic_vector(31 downto 0);  -- requested index
        result   : out std_logic_vector(31 downto 0);  -- F(N) (or truncated on ovf)
```

```
        done     : out std_logic;                        -- latched, cleared on next start
        overflow : out std_logic;                        -- latched, cleared on next start
        busy     : out std_logic;                        -- high while iterating
        cycles   : out std_logic_vector(31 downto 0)     -- iteration count (debug)
    );
end entity fib_core;

architecture rtl of fib_core is

    -- S_IDLE : waiting for start
    -- S_INIT : 1-cycle delay so done can de-assert reliably after start,
    --          even for the trivial n=0 / n=1 cases
    -- S_ADD  : iterate the add-and-shift datapath
    type state_t is (S_IDLE, S_INIT, S_ADD);
    signal state : state_t;

    signal a     : unsigned(31 downto 0);  -- F(k-1)
    signal b     : unsigned(31 downto 0);  -- F(k)
    signal k     : unsigned(31 downto 0);  -- current index we already have in b
    signal n_reg : unsigned(31 downto 0);  -- latched target

    signal result_r : std_logic_vector(31 downto 0);
    signal done_r   : std_logic;
    signal ovf_r    : std_logic;
    signal busy_r   : std_logic;
    signal cyc_r    : unsigned(31 downto 0);

    -- 33-bit extended adder so we can catch the carry-out
    signal sum_ext  : unsigned(32 downto 0);

begin

    sum_ext <= ('0' & a) + ('0' & b);

    result   <= result_r;
    done     <= done_r;
    overflow <= ovf_r;
    busy     <= busy_r;
    cycles   <= std_logic_vector(cyc_r);

    fsm : process (clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                state    <= S_IDLE;
                a        <= (others => '0');
                b        <= (others => '0');
                k        <= (others => '0');
                n_reg    <= (others => '0');
```

```
                result_r <= (others => '0');
                done_r   <= '0';
                ovf_r    <= '0';
                busy_r   <= '0';
                cyc_r    <= (others => '0');
            else
                case state is

                    when S_IDLE =>
                        busy_r <= '0';
                        if start = '1' then
                            n_reg  <= unsigned(n);
                            a      <= (others => '0');
                            b      <= (0 => '1', others => '0');
                            k      <= to_unsigned(1, 32);
                            ovf_r  <= '0';
                            done_r <= '0';   -- always drop done for >= 1 cycle
                            busy_r <= '1';
                            cyc_r  <= (others => '0');
                            state  <= S_INIT;
                        end if;

                    when S_INIT =>
                        -- handle the trivial cases here so that done went
                        -- through one full low cycle (seen by the whole system)
                        if n_reg = 0 then
                            result_r <= (others => '0');
                            done_r   <= '1';
                            busy_r   <= '0';
                            state    <= S_IDLE;
                        elsif n_reg = 1 then
                            result_r <= std_logic_vector(to_unsigned(1, 32));
                            done_r   <= '1';
                            busy_r   <= '0';
                            state    <= S_IDLE;
                        else
                            state <= S_ADD;
                        end if;

                    when S_ADD =>
                        cyc_r <= cyc_r + 1;

                        if sum_ext(32) = '1' then
                            -- overflow: latch truncated result and stop
                            ovf_r    <= '1';
                            result_r <= std_logic_vector(sum_ext(31 downto 0));
                            done_r   <= '1';
                            busy_r   <= '0';
                            state    <= S_IDLE;
                        elsif (k + 1) = n_reg then
```

```
                                -- this addition produces F(n_reg)
                                result_r <= std_logic_vector(sum_ext(31 downto
                                    0));
                                done_r   <= '1';
                                busy_r   <= '0';
                                state    <= S_IDLE;
                            else
                                a <= b;
                                b <= sum_ext(31 downto 0);
                                k <= k + 1;
                            end if;

                        end case;
                    end if;
            end if;
        end process fsm;

end architecture rtl;
```

Listing 1: `fib_core.vhd` — iterative Fibonacci FSM with overflow detection.

2.2 Test Bench and Block Diagram Snip

The self-checking testbench `tb_fib_core.vhd` exercises $N \in \{0, 1, 2, 5, 10, 20, 46, 47, 48\}$, covering both corner cases (N=0, N=1), the largest non-overflowing index (N=47, $F(47)$=2,971,215,073) and the first overflowing case (N=48). Figure 1 shows the Vivado `xsim` waveform: the `start` pulse, `busy`, `done` and the computed `result` bus for each case are visible, and the TCL console reports `TESTBENCH PASSED - all cases match`. Figure 2 shows the Vivado IP-integrator block diagram for the hardware build: the ZYNQ PS drives an AXI SmartConnect that fans out to the custom `fib_axi_0` IP.

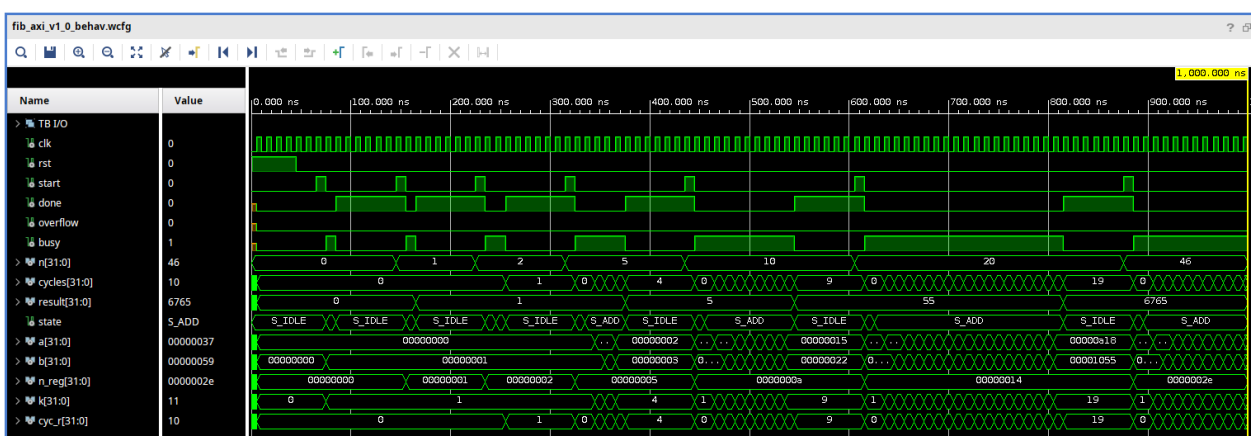

Figure 1: `tb_fib_core` simulation waveform in Vivado `xsim`. Each test case pulses `start`, raises `busy`, then asserts `done` with the matching $F(N)$ on the `result` bus. N=48 asserts `overflow` as expected.

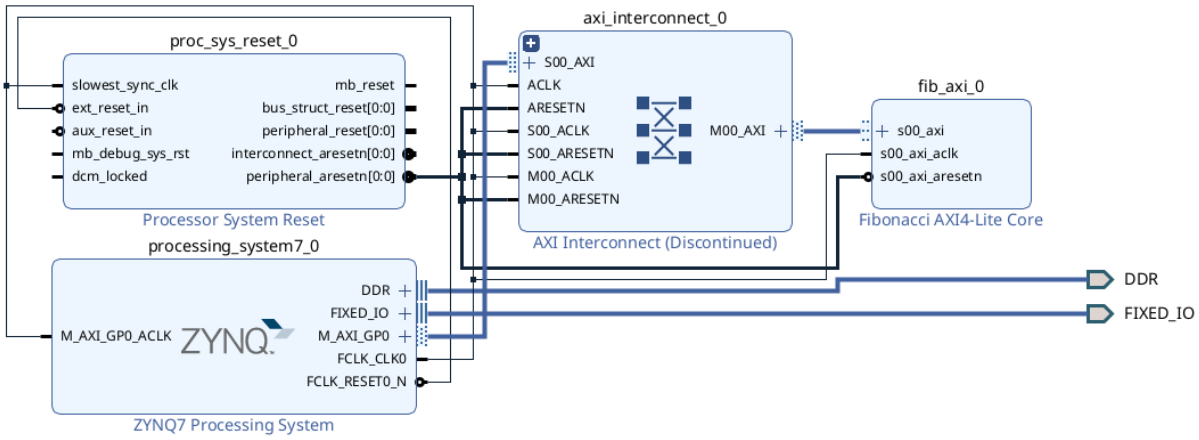

Figure 2: Vivado IP-integrator block design for the Fibonacci system. The ZYNQ PS drives the custom `fib_axi_0` IP through an AXI SmartConnect; all AXI slaves are synchronised to `FCLK_CLK0` via the processor system-reset block.

2.3 Test Application C Code

Listing 2 is the bare-metal Vitis application used to drive the IP on hardware. It writes N to `N_REG`, pulses the `START` bit in `CTRL_STAT`, polls the `DONE` flag and reads back $F(N)$, the `OVERFLOW` flag, and the iteration `CYCLES` counter. A local 64-bit software reference `sw_fib()` is cross-checked against every hardware result and a pass/fail column is emitted on the UART for easy screenshotting.

```c
/*
 * main.c
 *
 * ECEC 661 - Homework 3 - Problem 8.7.1 Fibonacci Number Generator
 *
 * Bare-metal test application for the custom "fib_axi" AXI4-Lite IP.
 * The PS drives N and the START bit, polls the DONE flag and reads
 * back F(N), the OVERFLOW flag and the iteration counter, then prints
 * every case over UART1 for PuTTY capture.
 *
 * Register map (byte offsets from XPAR_FIB_AXI_0_S00_AXI_BASEADDR):
 *   0x00  CTRL_STAT  W: [0] START          R: [1] DONE [2] OVERFLOW [3] BUSY
 *   0x04  N_REG      R/W target index
 *   0x08  FIB_REG    R/O F(N)
 *   0x0C  CYCLES     R/O iteration count
 */

#include <stdio.h>
#include "xparameters.h"
#include "xil_io.h"
#include "xil_printf.h"
#include "xil_types.h"
```

```c
#include "sleep.h"

/*
 * Base address.  Vitis generates XPAR_FIB_AXI_0_S00_AXI_BASEADDR when the
 * IP is present in the block design.  Fall back to the default 0x43C00000
 * slot so the file still compiles if the platform hasn't been built yet.
 */
#ifdef XPAR_FIB_AXI_0_S00_AXI_BASEADDR
#define FIB_BASE            XPAR_FIB_AXI_0_S00_AXI_BASEADDR
#else
#warning "XPAR_FIB_AXI_0_S00_AXI_BASEADDR not found - using default 0x43C00000"
#define FIB_BASE            0x43C00000U
#endif

#define FIB_CTRL_OFFSET     0x00U
#define FIB_N_OFFSET        0x04U
#define FIB_RESULT_OFFSET   0x08U
#define FIB_CYCLES_OFFSET   0x0CU

#define FIB_CTRL_START      (1U << 0)
#define FIB_STAT_DONE       (1U << 1)
#define FIB_STAT_OVERFLOW   (1U << 2)
#define FIB_STAT_BUSY       (1U << 3)

#define FIB_POLL_TIMEOUT    1000000U

/*
 * Software reference (64-bit) used to cross-check the hardware.
 */
static u64 sw_fib(u32 n)
{
    u64 a = 0, b = 1, t;
    for (u32 i = 0; i < n; i++) {
        t = a + b;
        a = b;
        b = t;
    }
    return a;
}

/*
```

```c
 * Drive a single compute on the HW.
 * ---------------------------------------------------------------------
 */
static u32 fib_compute(u32 n, int *overflow_out, u32 *cycles_out)
{
    Xil_Out32(FIB_BASE + FIB_N_OFFSET, n);
    Xil_Out32(FIB_BASE + FIB_CTRL_OFFSET, FIB_CTRL_START);

    u32 ctrl = 0;
    u32 timeout = FIB_POLL_TIMEOUT;
    do {
        ctrl = Xil_In32(FIB_BASE + FIB_CTRL_OFFSET);
    } while (!(ctrl & FIB_STAT_DONE) && --timeout);

    u32 result = Xil_In32(FIB_BASE + FIB_RESULT_OFFSET);
    if (cycles_out)   *cycles_out   = Xil_In32(FIB_BASE + FIB_CYCLES_OFFSET);
    if (overflow_out) *overflow_out = (ctrl & FIB_STAT_OVERFLOW) ? 1 : 0;

    if (timeout == 0) {
        xil_printf("   [!] timeout waiting for DONE (ctrl=0x%08x)\r\n", ctrl);
    }
    return result;
}

int main(void)
{
    const u32 test_n[] = { 0, 1, 2, 5, 10, 20, 30, 46, 47, 48 };
    const unsigned n_tests = sizeof(test_n) / sizeof(test_n[0]);

    /* startup window so the serial monitor can attach; also proves UART works
     * before any AXI access. Remove once UART output is confirmed. */
    for (int i = 3; i > 0; i--) {
        xil_printf("fib_gen starting in %d...\r\n", i);
        sleep(1);
    }

    xil_printf("\r\n");
    xil_printf("==========================================================\r\n");
    xil_printf("   ECEC 661 HW3 - Fibonacci AXI IP test application\r\n");
    xil_printf("   IP base address : 0x%08x\r\n", (unsigned) FIB_BASE);
    xil_printf("==========================================================\r\n");
    xil_printf("  n   |    fib(n) hex |      fib(n) dec |  ovf |  cycles | check\r\n");
    xil_printf("------+---------------+-----------------+------+---------+------\r\n");
```

```c

    unsigned pass = 0, fail = 0;

    for (unsigned i = 0; i < n_tests; i++) {
        u32 n      = test_n[i];
        int ovf    = 0;
        u32 cyc    = 0;
        u32 hwval  = fib_compute(n, &ovf, &cyc);
        u64 swval  = sw_fib(n);
        int sw_ovf = (swval > 0xFFFFFFFFULL) ? 1 : 0;

        int ok;
        if (sw_ovf) {
            ok = (ovf == 1);
        } else {
            ok = (ovf == 0) && (hwval == (u32) swval);
        }

        xil_printf("  %2u  | 0x%08x    |     %10u  |  %d   |  %5u  |  %s\r\n",
                (unsigned) n,
                (unsigned) hwval,
                (unsigned) hwval,
                ovf,
                (unsigned) cyc,
                ok ? "OK " : "BAD");

        if (ok) pass++; else fail++;

        /* small delay so the UART buffer is comfortable */
        usleep(10000);
    }

    xil_printf("\r\n");
    xil_printf("-----------------------------------------------------------\r\n");
    xil_printf("   Summary : %u passed, %u failed (total %u)\r\n",
               pass, fail, n_tests);
    xil_printf("-----------------------------------------------------------\r\n");

    while (1) {
        /* idle forever; the JTAG/UART session stays open for the snip */
    }

    return 0;
}
```

Listing 2: `fib_gen/sw/main.c` — bare-metal test driver for the Fibonacci AXI IP.

2.4 PuTTY Output

Figure 3 captures the UART console in PuTTY during a live run on the Cora Z7-07S board. The startup countdown proves that UART1 is alive before any AXI traffic, the per-row table lists each hardware result side-by-side with the `ok/bad` cross-check, and the summary line reports `10 passed, 0 failed`. N=48 correctly asserts `ovf=1` and the 32-bit truncated value is reported on its row.

```
OUTPUT ×   /DEV/TTYUSB1 (DIGILENT) ×   TASK: XSDB CONSOLE   DEBUG CONSOLE ×
Opened with baud rate: 115200

============================================================
   ECEC 661 HW3 - Fibonacci AXI IP test application
   IP base address : 0x43C00000
============================================================
  n   |    fib(n) hex |      fib(n) dec |  ovf |  cycles | check
------+---------------+-----------------+------+---------+------
   0  | 0x00000000    |               0 |  0   |       0 |  OK
   1  | 0x00000001    |               1 |  0   |       0 |  OK
   2  | 0x00000001    |               1 |  0   |       1 |  OK
   5  | 0x00000005    |               5 |  0   |       4 |  OK
  10  | 0x00000037    |              55 |  0   |       9 |  OK
  20  | 0x00001A6D    |            6765 |  0   |      19 |  OK
  30  | 0x000CB228    |          832040 |  0   |      29 |  OK
  46  | 0x6D73E55F    |      1836311903 |  0   |      45 |  OK
  47  | 0xB11924E1    |      2971215073 |  0   |      46 |  OK
  48  | 0x1E8D0A40    |       512559680 |  1   |      47 |  OK

------------------------------------------------------------
   Summary : 10 passed, 0 failed (total 10)
------------------------------------------------------------
```

Figure 3: PuTTY capture of the Fibonacci test application running on the Cora Z7-07S board. Every N matches the software reference; N=48 asserts the overflow flag as expected.

3 Design B — Fixed-Point Accumulator (`fix_acc`)

3.1 User-Logic HDL

The user-logic core is a thin wrapper around the Vivado IP-Catalog Accumulator `c_accum_0` (`xilinx.com:ip:c_accum:12.0`), kept to the book's canonical port signature `(B, CLK, SCLR, Q)` so that the slave-to-IP port map is a direct 1:1 pass-through. On every rising edge of `CLK` the accumulator commits $Q \leftarrow Q + B$, or a synchronous clear when `SCLR` is high (`SCLR` takes priority). The slave wrapper (`fix_acc_axi_v1_0_S00_AXI.vhd`) then wires its internal `slv_reg_wren` strobe to the core's `CLK`, following the textbook construction (*Digital Systems Projects*, §8.7.2), so that every AXI write is exactly one accumulate step. The register map is: `B_IN` at `0x00`, `Q_OUT` at `0x04`, and `CTRL` (SCLR) at `0x08`.

```vhdl
-- ------------------------------------------------------------------------------
-- fix_acc_core.vhd
--
-- ECEC 661 - Homework 3 - Problem 8.7.2 Fixed-Point Accumulator IP
--
-- Thin user-logic wrapper around the Vivado IP-Catalog Accumulator
--     xilinx.com:ip:c_accum:12.0
--
-- Operation:
--     On the rising edge of CLK, Q <= Q + B
--     When SCLR = '1', Q is synchronously zeroed (SCLR takes priority over add).
--     Output latency = 1 clock (Q is registered inside c_accum_0).
--
-- This entity is instantiated by the AXI4-Lite slave (fix_acc_axi_v1_0_S00_AXI),
-- where the slave write-enable strobe (slv_reg_wren) is wired to CLK so that
-- every AXI write to the IP advances the accumulator by exactly one step.
--
-- Ports kept at the book's signature (B/CLK/SCLR/Q) so the port map between the
-- slave and the IP-Catalog block is a direct 1:1 pass-through.
-- ------------------------------------------------------------------------------

library ieee;
use ieee.std_logic_1164.all;

entity fix_acc_core is
    port (
        B    : in  std_logic_vector(31 downto 0);  -- accumulator input
        CLK  : in  std_logic;                      -- driven by slv_reg_wren
        SCLR : in  std_logic;                      -- synchronous clear
        Q    : out std_logic_vector(31 downto 0)   -- accumulator output
    );
end entity fix_acc_core;

architecture rtl of fix_acc_core is

    -- Vivado IP-Catalog Accumulator v12.0 (black box; elaborated from XCI)
    component c_accum_0
        port (
            B    : in  std_logic_vector(31 downto 0);
            CLK  : in  std_logic;
            SCLR : in  std_logic;
            Q    : out std_logic_vector(31 downto 0)
        );
```

```vhdl
    end component;

begin

    U_ACC : c_accum_0
        port map (
            B    => B,
            CLK  => CLK,
            SCLR => SCLR,
            Q    => Q
        );

end architecture rtl;
```

Listing 3: `fix_acc_core.vhd` — user-logic wrapper around the Vivado IP-Catalog Accumulator.

3.2 Test Bench and Block Diagram Snip

The self-checking testbench `tb_fix_acc_core.vhd` reproduces Fig. 8.7.3 from the course text: after a synchronous clear, four clock cycles with $B{=}1$ produce $Q{=}1, 2, 3, 4$, and four more cycles with $B{=}{-2}$ produce $Q{=}2, 0, -2, -4$. Additional scenarios verify a mid-stream `SCLR`, $\sum_{i=1}^{10} i = 55$ (to match the software spot-check), and a large-magnitude accumulate. Figure 4 shows the resulting waveform in Vivado `xsim`; the console reports `TESTBENCH PASSED - all cases ok`. Figure 5 shows the IP-integrator block design for the hardware build, with the custom `fix_acc_axi_0` IP attached to the ZYNQ PS via an AXI SmartConnect.

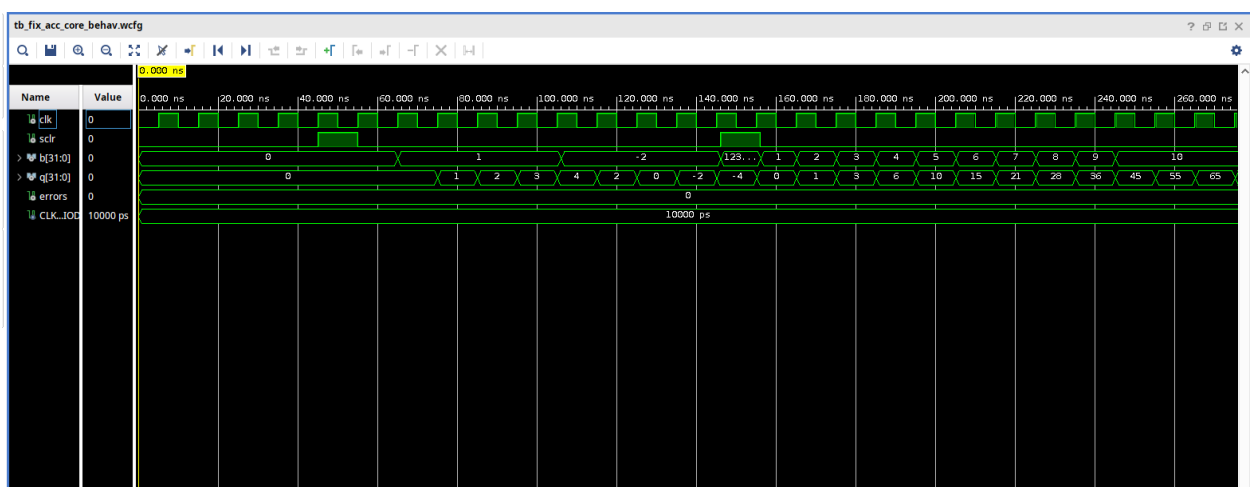

Figure 4: `tb_fix_acc_core` simulation waveform. The $B{=}1$ / $B{=}{-2}$ sequences match Fig. 8.7.3 in the textbook; the mid-stream `SCLR` pulse wipes Q back to zero.

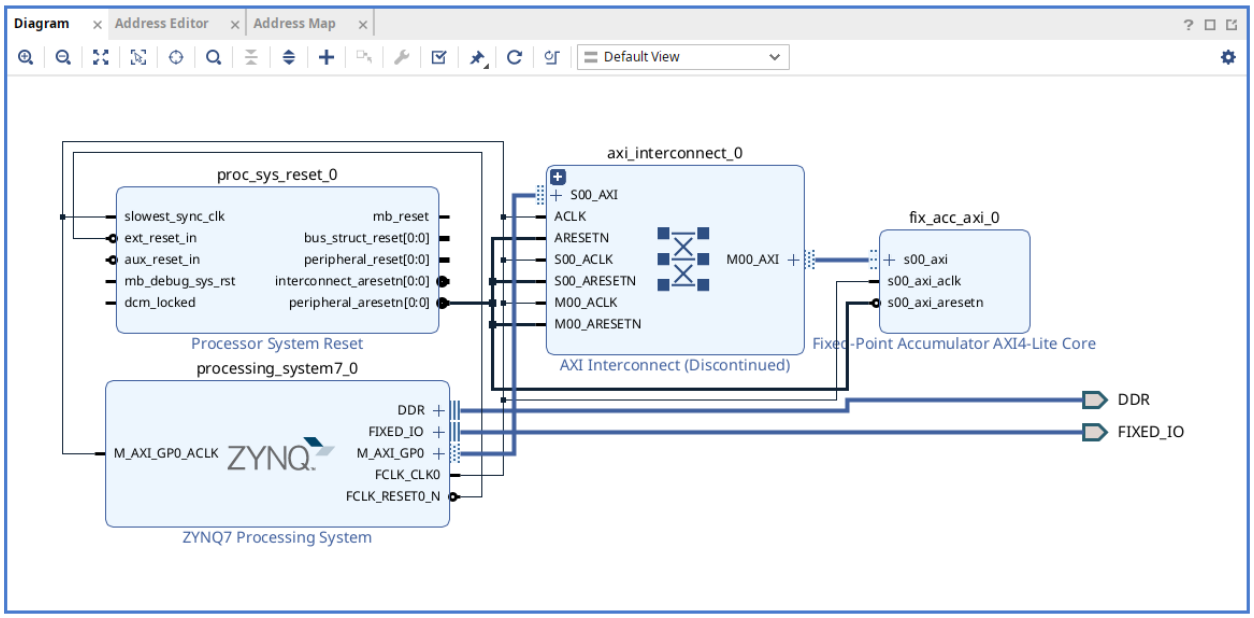

Figure 5: Vivado IP-integrator block design for the fixed-point accumulator system. The custom `fix_acc_axi_0` IP sits behind an AXI SmartConnect, sharing the `FCLK_CLK0` clock and reset with the ZYNQ PS.

3.3 Test Application C Code

Listing 4 is the bare-metal Vitis application used to drive `fix_acc_axi` on hardware. After a synchronous clear, the program reproduces the book table by writing $i = 0, \ldots, 9$ into `B_IN` and reading `Q_OUT` after each write, then extends the range to $i = 1{,}000$ and performs a latency-1 flush to show $Q = 500500 = \sum_{i=1}^{1000} i$. A battery of self-checks ($\sum_{i=1}^{10}$, $\sum_{i=1}^{100}$, $\sum_{i=1}^{1000}$, repeated negative accumulate, mid-stream `SCLR`) is then run with explicit pass/fail output on the UART.

```
/*
 * main.c
 *
 * ECEC 661 - Homework 3 - Problem 8.7.2 Fixed-Point Accumulator IP
 *
 * Bare-metal test application for the custom "fix_acc_axi" IP.  The PS
 * writes B-values to slv_reg0 (0x00) one at a time; each write pulses the
 * slave's write-enable strobe, which is wired to the IP-Catalog accumulator's
 * CLK, so each write is exactly one accumulate step.  A final "flush" write
 * is required to see the running sum on Q_OUT because the accumulator has
 * latency = 1.
 *
 * Register map (byte offsets from XPAR_FIX_ACC_AXI_0_S00_AXI_BASEADDR):
 *   0x00  B_IN   R/W  signed 32-bit input; writing advances Q by this value
 *   0x04  Q_OUT  R    signed 32-bit accumulator output
```

```c
 *   0x08  CTRL   R/W  [0]=SCLR (pulse 1 then 0 to zero Q)
 *   0x0C         --   reserved
 *
 * Test layout is inspired by fib_gen/sw/main.c so the report screenshots
 * match our HW3 Problem 8.7.1 aesthetic: 3 s startup countdown, header,
 * table of test rows, pass/fail summary.
 */

#include <stdio.h>
#include "xparameters.h"
#include "xil_io.h"
#include "xil_printf.h"
#include "xil_types.h"
#include "sleep.h"

/* ------------------------------------------------------------------------
 * Base address.  Vitis generates XPAR_FIX_ACC_AXI_0_S00_AXI_BASEADDR when
 * the IP is present in the block design.  Fall back to 0x43C10000 (the
 * address we assign in bd/fix_acc.bd) if the platform hasn't been built yet.
 * ------------------------------------------------------------------------
 */
#ifdef XPAR_FIX_ACC_AXI_0_S00_AXI_BASEADDR
#define FA_BASE            XPAR_FIX_ACC_AXI_0_S00_AXI_BASEADDR
#else
#warning "XPAR_FIX_ACC_AXI_0_S00_AXI_BASEADDR not found - using default 0x43C10000"
#define FA_BASE            0x43C10000U
#endif

#define FA_B_IN_OFFSET     0x00U
#define FA_Q_OUT_OFFSET    0x04U
#define FA_CTRL_OFFSET     0x08U

#define FA_CTRL_SCLR       (1U << 0)

/* ------------------------------------------------------------------------
 * Primitive helpers - all register-level work lives here.
 * ------------------------------------------------------------------------
 */
static inline void fa_write_b(s32 b)
{
    Xil_Out32(FA_BASE + FA_B_IN_OFFSET, (u32) b);
}

static inline s32 fa_read_q(void)
```

```c
{
    return (s32) Xil_In32(FA_BASE + FA_Q_OUT_OFFSET);
}

/*
 * Reset the accumulator to zero.  Sequence (from book p. 447):
 *   1. Write B_IN = 0 so that the "flush" pulse caused by writing CTRL
 *      doesn't contaminate Q with a leftover value from a previous run.
 *   2. Write CTRL = 1 (SCLR high).
 *   3. Write CTRL = 0 (SCLR low).
 */
static void fa_sclr(void)
{
    Xil_Out32(FA_BASE + FA_B_IN_OFFSET,  0U);
    Xil_Out32(FA_BASE + FA_CTRL_OFFSET,  FA_CTRL_SCLR);
    Xil_Out32(FA_BASE + FA_CTRL_OFFSET,  0U);
}

/*
    ---------------------------------------------------------------------------
 * Individual test scenarios.  Each returns 1 on pass, 0 on fail, and prints
 * a single table row so the UART capture reads like the fib_gen one.
 *
    ---------------------------------------------------------------------------
 */
/*
 * All table rows share this format so the header and data columns line up:
 *   "  <tag:18> | <N:7> | <expected:10> | <got:10> | <check>"
 */
#define FA_ROW_FMT  "  %-18s | %7u | %10d | %10d | %s\r\n"

static int test_sum_1_to_N(unsigned N, s32 expected, const char *tag)
{
    fa_sclr();
    for (unsigned i = 0; i <= N; i++) {
        fa_write_b((s32) i);
    }
    fa_write_b(0);                    /* flush the latency-1 pipeline */

    s32 got = fa_read_q();
    int ok  = (got == expected);

    xil_printf(FA_ROW_FMT, tag, N, (int) expected, (int) got,
               ok ? "OK " : "BAD");
    return ok;
}

static int test_negative_accum(void)
{
    fa_sclr();
```

```c
    for (int i = 0; i < 4; i++) {
        fa_write_b(-2);
    }
    fa_write_b(0);                      /* flush */

    s32 got      = fa_read_q();
    s32 expected = -8;                  /* -2 * 4 */
    int ok       = (got == expected);

    xil_printf(FA_ROW_FMT, "B=-2 four times", 4U,
               (int) expected, (int) got, ok ? "OK " : "BAD");
    return ok;
}

static int test_mid_stream_sclr(void)
{
    fa_sclr();
    for (int i = 1; i <= 5; i++) {
        fa_write_b(i);
    }
    fa_write_b(0);                      /* flush - Q now = 10 */

    fa_sclr();                          /* wipe it */

    /* after SCLR Q should be 0; a flush write is required to observe it */
    fa_write_b(0);

    s32 got      = fa_read_q();
    s32 expected = 0;
    int ok       = (got == expected);

    xil_printf(FA_ROW_FMT, "SCLR mid-stream", 5U,
               (int) expected, (int) got, ok ? "OK " : "BAD");
    return ok;
}

/*
 * main
 */
int main(void)
{
    /* Startup window so the serial monitor can attach; same idea as the
     * fib_gen app.  Remove once UART output is confirmed.
     */
    for (int i = 3; i > 0; i--) {
        xil_printf("fix_acc starting in %d...\r\n", i);
        sleep(1);
    }
```

```c

    xil_printf("\r\n");
    xil_printf("==========================================================\r\n");
    xil_printf("   ECEC 661 HW3 - Fixed-Point Accumulator IP test app\r\n");
    xil_printf("   IP base address : 0x%08x\r\n", (unsigned) FA_BASE);
    xil_printf("==========================================================\r\n");
    xil_printf("\r\n");

    /* ----------------------------------------------------------------------
     * Book reproduction.  Write i = 0 .. 10 into slv_reg0, read Q after each
     * write, and print the first ten rows.  This table is the near-identical
     * output shown on p. 447 of Digital Systems Projects.
     * ----------------------------------------------------------------------
     */
    xil_printf("Book reproduction (Digital Systems Projects, p. 447):\r\n");
    xil_printf("slv_reg0 |   Q\r\n");
    xil_printf("---------+-------\r\n");

    fa_sclr();
    for (int i = 0; i < 10; i++) {
        fa_write_b((s32) i);
        xil_printf("   %4d  | %6d\r\n", i, (int) fa_read_q());
    }
    /* finish the range and flush to match book's final print line */
    for (int i = 10; i <= 1000; i++) {
        fa_write_b((s32) i);
    }
    fa_write_b(0);  /* latency-1 flush */
    xil_printf("Last output, latency = 1: Q = %d (expected 500500)\r\n",
               (int) fa_read_q());

    xil_printf("\r\n");
    xil_printf("Self-checks:\r\n");
    xil_printf("  %-18s | %7s | %10s | %10s | %s\r\n",
               "scenario", "N", "expected", "got", "check");
    xil_printf("  ------------------"
               "-+-"  "-------"
               "-+-"  "----------"
               "-+-"  "----------"
               "-+-"  "-----" "\r\n");

    unsigned pass = 0, fail = 0;

    if (test_sum_1_to_N(10,   55,     "Sigma(1..10)"))    pass++; else fail++;
```

```c
    if (test_sum_1_to_N(100,  5050,   "Sigma(1..100)"))   pass++; else fail++;
    if (test_sum_1_to_N(1000, 500500, "Sigma(1..1000)"))  pass++; else fail++;
    if (test_negative_accum())                            pass++; else fail++;
    if (test_mid_stream_sclr())                           pass++; else fail++;

    xil_printf("\r\n");
    xil_printf("------------------------------------------------------------\r\n");
    xil_printf("   Summary : %u passed, %u failed (total %u)\r\n",
                pass, fail, pass + fail);
    xil_printf("------------------------------------------------------------\r\n");

    while (1) {
        /* idle forever; the JTAG/UART session stays open for the screenshot */
    }

    return 0;
}
```

Listing 4: `fix_acc/sw/main.c` — bare-metal test driver for the fixed-point accumulator AXI IP.

3.4 PuTTY Output

Figure 6 captures the UART console in PuTTY during a live run on the Cora Z7-07S board. The "book reproduction" block at the top matches p. 447 of the text exactly and reports the expected `Q = 500500` after the latency-1 flush; the self-check block below reports `5 passed, 0 failed`.

```
OUTPUT ×   /DEV/TTYUSB1 (DIGILENT) ×   DEBUG CONSOLE ×   PROBLEMS ×
Opened with baud rate: 115200
fix_acc starting in 3...
fix_acc starting in 2...
fix_acc starting in 1...

============================================================
   ECEC 661 HW3 - Fixed-Point Accumulator IP test app
   IP base address : 0x43C10000
============================================================

Book reproduction (Digital Systems Projects, p. 447):
slv_reg0 |   Q
---------+-------
      0  |      0
      1  |      0
      2  |      1
      3  |      3
      4  |      6
      5  |     10
      6  |     15
      7  |     21
      8  |     28
      9  |     36
Last output, latency = 1: Q = 500500 (expected 500500)

Self-checks:
  scenario           |       N |   expected |        got | check
  -------------------+---------+------------+------------+------
  Sigma(1..10)       |      10 |         55 |         55 | OK
  Sigma(1..100)      |     100 |       5050 |       5050 | OK
  Sigma(1..1000)     |    1000 |     500500 |     500500 | OK
  B=-2 four times    |       4 |         -8 |         -8 | OK
  SCLR mid-stream    |       5 |          0 |          0 | OK

------------------------------------------------------------
   Summary : 5 passed, 0 failed (total 5)
------------------------------------------------------------
```

Figure 6: PuTTY capture of the fixed-point accumulator test application. The accumulator reproduces the book table, reaches Q=500500 after the latency-1 flush, and passes every self-check scenario.

4 Conclusion

Two custom AXI4-Lite IP blocks were designed, packaged and verified end-to-end on the Digilent Cora Z7-07S board. The Fibonacci IP uses an iterative add-and-shift FSM with 33-bit overflow detection and was exercised over its full useful range ($N = 0$ to 48), including

the first overflowing index. The fixed-point accumulator IP wraps the Vivado IP-Catalog `c_accum_0` and exploits the slave's write-enable strobe as the accumulator clock, so that every AXI write advances Q by exactly one step; it reproduces the textbook simulation figure and the $\sum_{i=1}^{1000} i = 500500$ check in both simulation and hardware. For each design, the user-logic HDL source, the simulation/block-diagram evidence, the bare-metal C test application, and the PuTTY UART capture have been supplied alongside this report, satisfying the homework deliverables.